



**NEXUS ENGENHARIA DE TI
SOFTWARE PARA CONTROLE DE REQUISITOS**

Eduardo Lucas Lemes Januário

Guilherme Batista de Souza

Lucas Marcelino

Ruan Vinicius Almagro

CATANDUVA

2025

NEXUS ENGENHARIA DE TI

“Controle de Qualidade não é um departamento, é uma atitude”

- *Kaoru Ishikawa*



Resumo

Este trabalho apresenta o desenvolvimento conceitual de um Sistema de Gestão e Versionamento de Requisitos (SGR) para a empresa Algomith Software, elaborado pela Nexus Engenharia de TI. O sistema tem como propósito centralizar, versionar e reutilizar requisitos de software, aplicando princípios de rastreabilidade e governança. Baseado em conceitos de versionamento semelhantes ao Git, o SGR garante que cada requisito seja tratado como um artefato versionável, com histórico de mudanças, autoria e justificativa. Além disso, incorpora inteligência artificial para auxiliar na detecção de duplicidades, reformulação textual, sugestão de critérios de aceite e apoio à reutilização. O modelo de processo adotado é evolucionário, iterativo e incremental, com metodologias ágeis inspiradas no Scrum e no Extreme Programming, permitindo entregas contínuas e de alto valor. A qualidade é tratada como eixo estratégico, fundamentada em normas como a ISO/IEC 25010. O estudo reforça a importância da rastreabilidade, da automação e da inteligência artificial no gerenciamento de requisitos, promovendo eficiência, consistência e padronização no ciclo de desenvolvimento de software.

Palavras-chave: requisitos de software; versionamento; rastreabilidade; inteligência artificial; qualidade de software.

Abstract

This paper presents the conceptual development of a Requirements Management and Versioning System (SGR) for the company Algomith Software, designed by Nexus Engenharia de TI. The system aims to centralize, version, and reuse software requirements by applying traceability and governance principles. Based on versioning concepts similar to Git, the SGR ensures that each requirement is handled as a versionable artifact, with a clear history of changes, authorship, and justification. It also incorporates artificial intelligence to support duplication detection, text reformulation, acceptance criteria suggestions, and reuse recommendations. The adopted process model is evolutionary, iterative, and incremental, supported by agile methodologies inspired by Scrum and Extreme Programming, enabling continuous and high-value deliveries. Quality is treated as a strategic pillar, grounded on standards such as ISO/IEC 25010. The study highlights the importance of traceability, automation, and artificial intelligence in requirements management, fostering efficiency, consistency, and standardization within the software development lifecycle.

Keywords: software requirements; versioning; traceability; artificial intelligence; software quality.

Índice

Introdução	5
Problemática	5
A Empresa – Nexus Engenharia de TI	5
Sistema de Gestão e Versionamento de Requisitos (SGR)	5
Fluxo Principal do Projeto	6
Modelo de Processo de Desenvolvimento	9
Metodologia Ágil de Trabalho – MAO	10
Funções no Orbit	10
Princípios norteadores da MAO	11
Estrutura de trabalho	11
Integração entre princípios e diferenciais	12
Decomposição de partes entregáveis	12
Qualidade do Software	12
Controle e verificação da qualidade	13
Especificação da qualidade	13
Metodologia de qualidade adotada	13
Áreas de qualidade priorizadas	14
Documentação	14
Conclusão	15
Referências	16

Introdução

O cenário atual do desenvolvimento de software exige das empresas rapidez, organização e eficiência no gerenciamento de requisitos. Grandes companhias que lidam com múltiplos clientes enfrentam o desafio de documentar, versionar e reaproveitar requisitos de maneira clara e estruturada. Nesse contexto, a Algomith Software, uma empresa consolidada no mercado de tecnologia e responsável por atender inúmeros clientes em paralelo, identificou a necessidade de implantar um sistema especializado para o controle e versionamento de requisitos.

Reconhecendo a importância dessa iniciativa e a complexidade envolvida, a Algomith optou por solicitar à Nexus Engenharia de TI o desenvolvimento de um projeto conceitual para esse sistema. A parceria estabelecida entre as duas empresas garante que a solução seja construída com base em experiência prática, inovação tecnológica e metodologias ágeis.

Problemática

A Algomith Software, por atender uma vasta carteira de clientes, lida diariamente com uma alta quantidade de solicitações de software que compartilham requisitos semelhantes. Sem uma ferramenta específica de rastreabilidade, versionamento e reuso, esses requisitos acabam se tornando duplicados, inconsistentes ou mal documentados, prejudicando a produtividade e a qualidade dos sistemas entregues.

O principal desafio identificado é que, apesar de ser uma empresa de grande porte e referência no setor, a Algomith não consegue dedicar sua própria equipe para o desenvolvimento dessa solução, já que sua prioridade é manter os projetos ativos para seus clientes. Assim, a terceirização desse software torna-se essencial para garantir que a gestão de requisitos passe a ser tratada de forma estruturada, eficiente e escalável.

A Empresa – Nexus Engenharia de TI

Para atender essa demanda, a Nexus Engenharia de TI, empresa de médio porte consolidada no mercado, foi a escolhida para propor o modelo conceitual do sistema de versionamento de requisitos.

A Nexus destaca-se por sua experiência em projetos que combinam inovação, qualidade e foco no cliente. Ao longo de sua trajetória, já desenvolveu soluções robustas como softwares de gerenciamento de bancos de dados, sistemas de ponto de venda (PDVs) e plataformas de gestão empresarial, sempre com alto padrão de qualidade e confiabilidade.

A empresa também mantém uma forte parceria com o Instituto Federal de São Paulo (IFSP) – Campus Catanduva, promovendo integração entre o mercado de trabalho e a formação acadêmica por meio de estágios. Foi justamente dessa colaboração que surgiu a oportunidade de conhecer de perto a necessidade da Algomith Software.

A Nexus, então, reuniu sua equipe de especialistas e estudantes em Engenharia de Software para propor um Sistema de Gestão de Requisitos (SGR) moderno, evolutivo e baseado em metodologias ágeis. Esse projeto conceitual não apenas atende às necessidades da Algomith, mas também reflete a expertise da Nexus em entregar soluções sob medida, prezando pela rastreabilidade, reuso e qualidade de software.

Sistema de Gestão e Versionamento de Requisitos (SGR)

O Sistema de Gestão e Versionamento de Requisitos (SGR) nasce como uma solução

estratégica para transformar a maneira como a Algomith Software organiza, documenta e reutiliza seus requisitos. A ideia central é tratar cada requisito como um artefato versionável, de forma muito semelhante ao que ocorre em plataformas de controle de código como o GitHub, mas adaptado à realidade da engenharia de software focada em requisitos. Assim como em um repositório de código, os requisitos poderão ser criados, modificados, versionados e armazenados em um histórico confiável, garantindo que cada mudança seja registrada com autor, justificativa e data, além de permitir que diferentes projetos se beneficiem de um mesmo conjunto de especificações sem que se perca a rastreabilidade de suas origens.

O funcionamento do sistema baseia-se na ideia de commits e branches. Cada vez que um requisito é criado, ele é salvo como uma versão única, podendo ser reutilizado em novos projetos ou adaptado de acordo com as necessidades de cada cliente. Quando ocorre a necessidade de individualização, ou seja, a adaptação de um requisito geral para atender a um caso específico, o sistema cria uma nova ramificação, preservando a ligação com o requisito de origem e registrando essa modificação como parte de um versionamento contínuo. Esse modelo garante não apenas a consistência entre projetos, mas também promove a rastreabilidade completa das alterações, oferecendo uma linha do tempo clara e auditável de todas as decisões tomadas.

O diferencial do SGR está no uso intensivo de inteligência artificial como suporte ao analista de requisitos. Enquanto novos requisitos são digitados, a IA atua como um “autocomplete inteligente”, verificando automaticamente toda a base já existente para identificar itens semelhantes ou duplicados. Caso encontre correspondências relevantes, o sistema sugere reaproveitar ou adaptar requisitos já salvos, evitando retrabalho e incentivando a padronização. Mesmo quando não há registros parecidos, a IA pode propor rascunhos iniciais com base em padrões de requisitos de domínio, além de revisar a clareza e a qualidade da descrição, sugerindo melhorias na redação, inclusão de critérios de aceite e eliminação de ambiguidades.

Essa combinação entre versionamento estruturado e suporte cognitivo da IA cria um ecossistema robusto, capaz de evoluir constantemente. A cada commit, não apenas se constrói um histórico de requisitos, mas também se alimenta um repositório vivo, no qual o conhecimento da empresa é consolidado e disponibilizado para futuras demandas. A analogia com o Git não é apenas conceitual: a proposta é dar à Algomith a mesma segurança e eficiência que equipes de desenvolvimento têm ao versionar código, mas agora aplicada ao coração dos projetos de software, que são os requisitos. Dessa forma, o SGR se apresenta como uma solução completa, promovendo rastreabilidade, reuso inteligente e qualidade contínua em todos os níveis do processo de desenvolvimento.

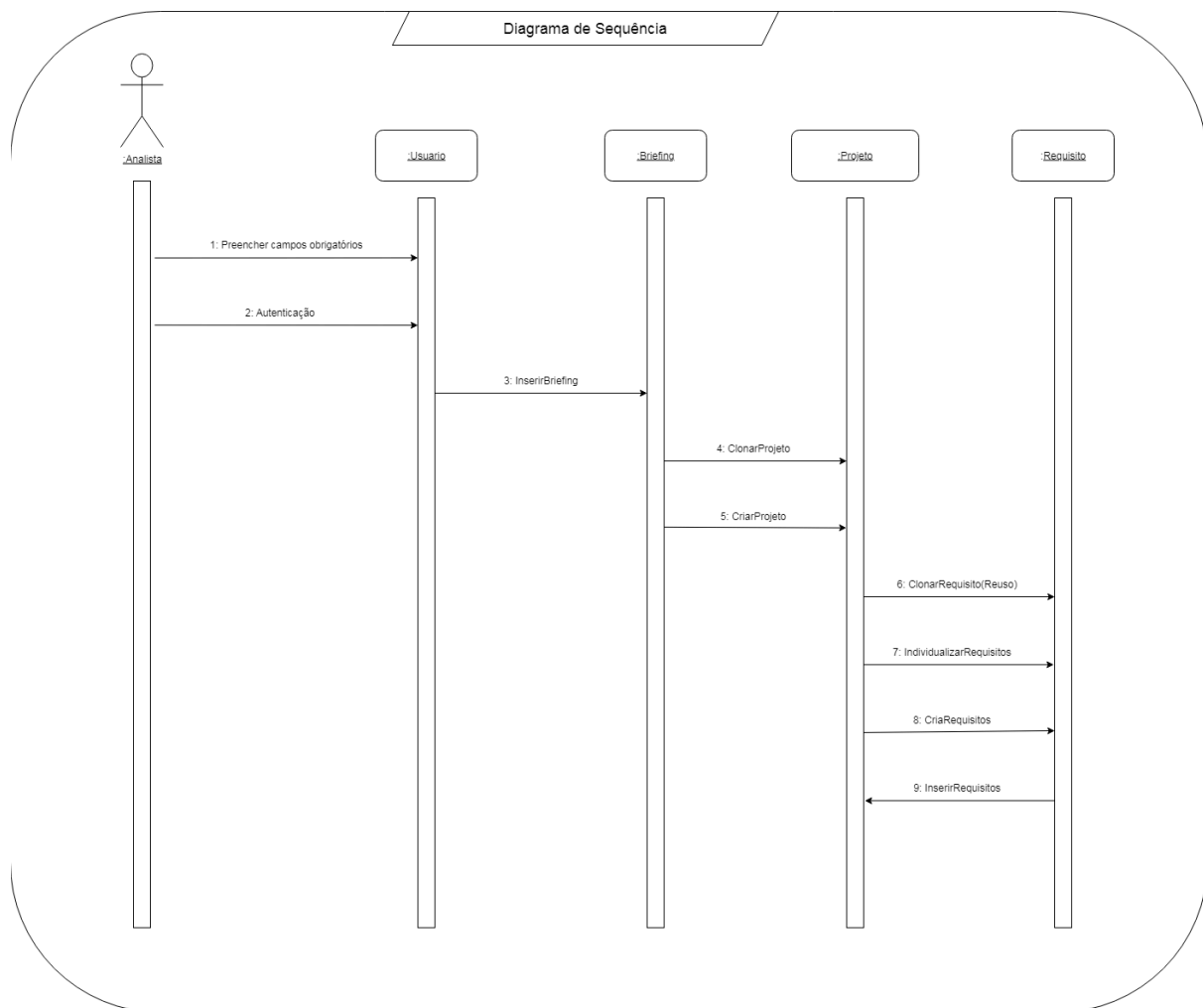
Fluxo Principal do Projeto

O fluxo principal do Sistema de Gerenciamento de Requisitos (SGR) representa a espinha dorsal do funcionamento da solução proposta, sendo a sequência de atividades essenciais que conduzem desde a autenticação inicial do usuário até o registro definitivo dos requisitos dentro do projeto. Esse fluxo concentra as interações fundamentais entre os atores e os módulos do sistema, traduzindo como o processo de elicitação, reuso e salvamento de requisitos ocorre de maneira prática e controlada.

No contexto do projeto como um todo, o fluxo principal assume papel estratégico: é nele que se materializam os conceitos centrais de versionamento, rastreabilidade e reuso, pilares da proposta apresentada pela Nexus Engenharia de TI para atender às demandas da Algomith. Cada etapa, desde a criação de um novo projeto e o registro do briefing inicial até a sugestão de requisitos pela inteligência artificial e o salvamento definitivo com rastreabilidade completa, foi pensada para garantir governança, consistência e qualidade.

O diagrama de sequência foi escolhido como recurso de representação justamente por oferecer clareza visual e precisão quanto à ordem dos eventos e às responsabilidades envolvidas. Ele permite observar, de forma linear e estruturada, como os módulos do sistema interagem e como o analista conduz as atividades ao longo do processo. Essa escolha facilita não apenas a compreensão técnica pelos desenvolvedores, mas também a comunicação com gestores e stakeholders, que conseguem visualizar de maneira objetiva o caminho percorrido por um requisito dentro do sistema.

Portanto, o fluxo principal não é apenas um processo técnico descrito; ele é a tradução prática da proposta de valor do SGR, demonstrando como a solução resolve os problemas de redundância, falta de controle e dificuldade de rastreabilidade enfrentados pela Algomith. Ao mesmo tempo, o diagrama associado atua como ferramenta de documentação e validação, assegurando que todos os envolvidos tenham uma visão comum e compartilhada sobre o funcionamento do sistema.



O Analista acessa o SGR e, na tela de autenticação, preenche os campos obrigatórios (usuário/e-mail e senha; quando habilitado, MFA). Ao confirmar, o módulo :Usuário valida credenciais, abre a sessão e registra a trilha de auditoria (IP, horário, agente). Com sucesso, o sistema encaminha o Analista diretamente para a interface de Projetos, aplicando as permissões do perfil.

Na sequência, o Analista parte para o briefing. No módulo :Briefing, ele insere o resumo das primeiras reuniões com o cliente (objetivos de negócio, escopo macro, restrições, stakeholders, prazos e métricas). Assim que o conteúdo é submetido (mensagem InserirBriefing), a IA indexa o texto e

inicia uma varredura semântica no acervo corporativo para encontrar projetos semelhantes. Se a similaridade superar o limiar configurado, o sistema recomenda ClonarProjeto: cria-se um novo contexto a partir do projeto mais aderente, herdando estrutura (taxonomia de requisitos, templates, convenções de nomenclatura, políticas de mudança) e mantendo vínculo de origem para rastreabilidade. Se não houver projeto compatível, o SGR automaticamente aciona CriarProjeto, provisionando um workspace novo com identificador único, taxonomia padrão e junção explícita do briefing ao projeto recém-criado. Em ambos os casos, o briefing fica relacionado ao projeto e disponível como contexto ativo para a IA.

A tela de Projetos possui recursos de filtro, busca e alteração (fora do escopo deste diagrama), mas, para este fluxo, focamos no cadastro guiado. Após clonar ou criar, o módulo :Projeto conclui a preparação do workspace e entrega ao Analista uma visão inicial curada pela IA: um painel com sugestões de requisitos coerentes com o briefing e (quando houve clonagem) com o histórico do projeto de origem. Essa curadoria antecipa o trabalho do Analista na próxima etapa, reduzindo retrabalho e padronizando o ponto de partida do backlog.

Então o Analista segue para Requisitos. Ao abrir o editor de requisitos no :Requisito, a IA lê o briefing vinculado e executa uma busca semântica na base corporativa, retornando sugestões coerentes com o contexto. As sugestões vêm com score de similaridade, trechos destacados e ações rápidas. A partir daqui, o Analista pode seguir três caminhos, todos previstos no diagrama:

- Quando há um requisito muito semelhante ao objetivo do projeto, o Analista escolhe clonar. O sistema traz o conteúdo e metadados do requisito corporativo para o projeto, preservando o vínculo de origem (para rastreabilidade e futura análise de impacto). Antes do salvamento, a IA roda checks de duplicidade e o ReqLint (clareza, completude, critérios de aceite, ausência de termos vagos). Em conformidade, o Analista segue para salvar.
- Se o requisito sugerido precisa de ajustes ao contexto do cliente, o Analista individualiza: edita título/descrição/escopo/critério de aceite, registra justificativa e marcações (ex.: variações por legislação, SLAs, integrações). A IA ajuda com reformulações para eliminar ambiguidades e pode sugerir critérios de aceite testáveis. O editor mostra diff por seção para tornar explícitas as personalizações. Passando nos checks (qualidade/duplicidade), o fluxo avança ao salvamento.
- Se a IA não encontrar requisitos adequados, ela ainda propõe ideias de novos requisitos com base no briefing (padrões de login, auditoria, exportação, etc.). O Analista cria o requisito do zero, preenchendo os campos estruturados (título, descrição, justificativa, origem, prioridade, riscos, critérios de aceite). O ReqLint verifica coerência entre descrição e critérios, pede ajustes quando necessário e, se tudo estiver consistente, o requisito fica pronto para persistência.

Antes do salvamento, o SGR executa os checks automáticos: duplicidade na base (com sugestão de unificação ou especialização, se necessário) e o ReqLint (clareza, completude de metadados, consistência entre descrição e critérios, ausência de termos vagos). Estando conforme, o Analista confirma InserirRequisitos; o :Requisito gera um commit com hash, autor, timestamp e mensagem padronizada (incluindo justificativa), vincula cada requisito ao projeto e ao briefing e atualiza a linha do tempo. Quando houve reuso, os vínculos de origem ficam registrados para rastreabilidade e análise de impacto futura. O Analista recebe a confirmação e pode continuar o cadastramento, agora com a IA aprendendo com as escolhas feitas para refinar próximas sugestões.

Modelo de Processo de Desenvolvimento

Para a construção do Sistema de Gestão e Versionamento de Requisitos (SGR), adotaremos um Modelo de Processo Evolucionário, Iterativo e Incremental. Essa escolha não é casual: ela reflete tanto a complexidade do sistema quanto a necessidade da Algomith de ter entregas frequentes, funcionais e que possam ser avaliadas em tempo real. Diferente de modelos tradicionais, que buscam entregar o produto apenas ao final de um longo ciclo, o processo adotado garante que o software cresça gradualmente, consolidando aprendizados a cada etapa.

O caráter evolucionário do processo significa que o SGR não será concebido como um produto fechado, mas como uma solução que amadurece ao longo do tempo, evoluindo conforme a Algomith utiliza os incrementos entregues e fornece feedback. Isso garante que o sistema acompanhe as mudanças naturais do ambiente de negócio, adaptando-se a novos requisitos, tecnologias ou prioridades.

A dimensão iterativa traduz-se em ciclos curtos e repetidos, nos quais a equipe da Nexus realiza atividades de planejamento, modelagem, implementação, testes e validação. A cada iteração, é possível revisar decisões, corrigir rotas e aprimorar funcionalidades. Por exemplo, uma primeira iteração pode focar no núcleo de cadastro e login de usuários, enquanto uma segunda pode evoluir para o commit e versionamento básico de requisitos, seguida por ciclos que adicionam a IA de similaridade, o controle de branches e, posteriormente, os relatórios de rastreabilidade.

O aspecto incremental garante que cada ciclo entregue um pedaço do sistema que seja funcional e utilizável, ainda que não definitivo. Esses incrementos são projetados para se integrar ao todo, formando gradualmente um sistema robusto. Cada incremento é, portanto, um bloco de valor agregado que pode ser testado pela Algomith, permitindo que o cliente perceba resultados práticos desde os primeiros estágios do projeto. Esse formato reduz riscos, já que eventuais problemas são identificados e corrigidos rapidamente, sem comprometer o projeto inteiro.

Além disso, o processo prevê que feedback contínuo seja parte essencial da construção. Cada incremento entregue é avaliado pelo Product Owner da Algomith e pela equipe envolvida, de modo que sugestões, ajustes e correções são absorvidos no ciclo seguinte. Essa retroalimentação transforma a relação cliente-desenvolvedor em uma parceria ativa, garantindo que o sistema final seja não apenas tecnicamente sólido, mas também adequado às necessidades reais da empresa.

Em termos práticos, o modelo de processo aplicado ao SGR pode ser ilustrado assim:

1. Ciclo 1: Autenticação, cadastro de usuários e criação de workspaces de projeto.
2. Ciclo 2: Elicitação de requisitos, commits iniciais e salvamento no banco de dados.
3. Ciclo 3: Implementação da IA de autocomplete e verificação de duplicidade.
4. Ciclo 4: Controle de branches e mecanismos de merge.
5. Ciclo 5: Baselines, relatórios de rastreabilidade e exportações.
6. Ciclos posteriores: Otimizações, integrações externas (Jira, Git), métricas de reuso e dashboards executivos.

Cada ciclo mantém as atividades fundamentais: planejamento do que será feito, modelagem conceitual do incremento, desenvolvimento, testes automatizados e revisão com o cliente. Ao final, uma nova versão do sistema, estável e utilizável, é disponibilizada. Assim, o produto não nasce pronto de uma vez, mas cresce organicamente, reduzindo riscos, maximizando valor e promovendo adaptação contínua.

Metodologia Ágil de Trabalho – MAO

A Nexus Engenharia de TI optou por aplicar, no desenvolvimento do SGR, a Metodologia Ágil Orbit (MAO), um framework original concebido pela própria equipe. Essa metodologia foi idealizada para responder às demandas específicas da Algomith, cliente que necessita não apenas da entrega contínua de novas funcionalidades, mas também da constante renovação do que já foi desenvolvido. A MAO organiza o trabalho em órbitas de valor, que funcionam como ciclos curtos, previsíveis e orientados por objetivos centrais definidos pelo cliente. Em cada órbita, o sistema avança de forma expansiva, agregando novas implementações, mas também retorna ao núcleo de valor já consolidado, revisando, refinando e garantindo a consistência do que foi entregue anteriormente. Essa dinâmica transforma o processo de desenvolvimento em um movimento orbital, no qual cada entrega mantém ligação direta com o centro gravitacional do projeto — o cliente — assegurando evolução constante e controle estratégico.

A justificativa para a adoção da MAO decorre da necessidade de lidar com três grandes desafios identificados no contexto do SGR: o elevado volume de requisitos a serem geridos, a obrigatoriedade de garantir reuso e versionamento eficazes, e a manutenção contínua da qualidade. Embora metodologias tradicionais como o Scrum ou o XP tenham méritos significativos na organização de ciclos iterativos, elas carecem de mecanismos formais para a revisão sistemática de funcionalidades já entregues. No caso do SGR, essa característica é vital, pois o sistema precisa não apenas expandir, mas também se renovar constantemente para evitar redundâncias, obsolescência e falhas de rastreabilidade. A metáfora orbital criada pela Nexus nasce dessa lacuna: unir entrega incremental a um processo cíclico de revisão, o que garante ao cliente estabilidade e inovação simultaneamente.

Funções no Orbit

A MAO estrutura-se em quatro funções complementares e interdependentes, responsáveis por manter a estabilidade do fluxo orbital.

O Sun é o cliente — no caso, a Algomith — e representa o centro de gravidade do processo. Todas as decisões e prioridades partem dele, sendo o eixo em torno do qual as atividades da equipe gravitam.

Os Satellites são os membros da equipe que trabalham diretamente no desenvolvimento técnico, implementando funcionalidades, melhorias e ajustes de acordo com as necessidades

s orbitais. Nesta equipe, Lucas atua como responsável pelo núcleo de integrações e versionamento de requisitos, assegurando rastreabilidade e consistência em todos os registros. Guilherme, por sua vez, dedica-se à implementação de funcionalidades orientadas pela IA, cuidando especialmente da interação entre briefing, sugestão de requisitos e salvamento no sistema.

O Orbit Master é o guardião da metodologia. No caso, Eduardo assume essa função, garantindo que o fluxo orbital se mantenha estável, que os princípios da MAO sejam respeitados e que a equipe esteja sempre alinhada com os objetivos definidos.

O Gravity Mediator, desempenhado por Ruan, tem o papel essencial de capturar, organizar e traduzir o feedback do cliente em backlog acionável, assegurando que nada se perca no processo de comunicação e que os requisitos técnicos estejam sempre conectados às demandas de negócio. Essa

função reforça a ponte entre a Algomith e a equipe de desenvolvimento, evitando desvios e mal-entendidos.

Essa configuração garante que cada analista da Nexus tenha responsabilidades claras e bem definidas, promovendo sinergia, foco e alinhamento contínuo entre as entregas técnicas e os objetivos de negócio do cliente.

Princípios norteadores da MAO

A MAO é sustentada por quatro princípios que asseguram o equilíbrio entre entrega de valor e manutenção contínua:

O primeiro é a Ciclicidade de valor real, que garante que, a cada órbita de duas semanas, a Algomith receba não apenas novas funcionalidades, mas também entregas revisadas e renovadas. Essa característica evita a estagnação do sistema, já que todo ciclo inclui um “Return to the Sun” — um retorno ao núcleo central — para validar e manter a relevância do que já foi produzido.

O segundo princípio é a Colaboração com o cliente, que assume protagonismo no papel de Sun. A participação ativa da Algomith garante que os objetivos de negócio sejam constantemente revisitados e validados, reduzindo ruídos de comunicação e assegurando alinhamento contínuo.

O terceiro princípio é a Renovação contínua, pois em cada órbita parte do esforço é direcionado ao refinamento de entregas anteriores. Isso garante evolução sem acúmulo de dívidas técnicas, além de permitir ajustes naturais frente a mudanças de escopo ou contexto.

O quarto princípio é a Medição de progresso baseada em software funcional e validado. Aqui, não basta contabilizar novas funcionalidades; é igualmente importante mensurar o valor mantido, ou seja, verificar se o que já existe continua útil, rastreável e consistente.

Estrutura de trabalho

A estrutura da MAO é organizada em ciclos de órbitas quinzenais, cada um dividido em três fases:

- Alignment: definição clara dos objetivos da órbita e priorização de entregas.
- Execution: implementação de novas funcionalidades acompanhada de revisões em entregas passadas.
- Return to the Sun: validação final junto ao cliente, assegurando que tudo esteja em órbita de valor.

Cada órbita inicia-se com o Planejamento, onde são definidos os objetivos de valor. Por exemplo: na Órbita 1, login e autenticação; na Órbita 2, versionamento de requisitos; na Órbita 3, sugestões inteligentes da IA.

Durante as órbitas, ocorrem os Diários Orbitais, encontros curtos de 15 minutos para alinhar status, remover bloqueios e ajustar o curso do trabalho. Ao final, realiza-se a Revisão da Órbita, onde

as entregas são apresentadas ao cliente, e a Retrospectiva Orbital, onde a equipe avalia sua própria performance, identificando desvios e melhorias.

Integração entre princípios e diferenciais

A força da MAO está na união de seus princípios com diferenciais práticos. A ciclicidade de valor real garante que as entregas não apenas cresçam, mas também se renovem. A colaboração ativa com o cliente fortalece a validação contínua e reduz falhas de interpretação. A renovação constante assegura evolução orgânica, evitando dívidas técnicas. E a métrica de progresso centrada no valor mantido assegura que a qualidade seja transversal a todo o processo.

Diferentemente de frameworks como o Scrum, que se concentram em entregas lineares, ou de abordagens híbridas que apenas adaptam ciclos curtos, a MAO inova ao garantir que cada órbita seja simultaneamente expansiva e revisora. Esse diferencial faz com que o sistema não apenas avance, mas também amadureça a cada iteração, consolidando um processo robusto, estável e adaptável às necessidades da Algomith.

Decomposição de partes entregáveis

Seguindo a lógica orbital e o fluxo principal do SGR, as entregas foram decompostas em cinco órbitas:

- Órbita 1: módulo de autenticação e cadastro de projetos, criando a base estrutural.
- Órbita 2: módulo de briefing e indexação de conteúdo pela IA, fornecendo contexto ao fluxo e realizando a busca de projetos similares já existentes, possibilitando o reuso integral do projeto por clonagem e versionamento, o reuso parcial com sugestões de adaptações ou a criação de um novo projeto quando não houver correspondência.
- Órbita 3: módulo de requisitos, com IA atuando como motor de sugestões (clonar, individualizar, criar novos), como aplicado na órbita 2 para o projeto, porém para os requisitos individuais de cada um dos projetos.
- Órbita 4: governança de mudanças e exportação de requisitos (ReqIF, JSON, PDF), consolidando o ciclo com integração externa.

Essa decomposição assegura que cada parte seja construída de forma incremental, validada em conjunto com o cliente e revisada em órbitas subsequentes. Assim, a MAO garante não apenas entregas contínuas, mas também renovação constante de valor, estabelecendo-se como metodologia original e estratégica para o desenvolvimento do SGR.

Qualidade do Software

A qualidade do SGR será tratada como um pilar estratégico de desenvolvimento, não apenas como uma etapa final de verificação. Nossa visão é que a qualidade deve estar presente desde o planejamento até a manutenção, garantindo que o sistema atenda às expectativas da Algomith em termos de confiabilidade, segurança, eficiência e facilidade de uso.

Controle e verificação da qualidade

A Nexus Engenharia de TI adotará um ciclo contínuo de prevenção, avaliação e correção, baseado nos conceitos apresentados por Pressman e nas normas ISO 9126. A qualidade será monitorada por meio de métricas objetivas e feedback prático do cliente em cada incremento entregue. Os principais mecanismos serão:

- Testes automatizados e manuais: garantindo que cada commit de funcionalidade seja validado contra requisitos funcionais e não funcionais.
- Revisões técnicas e inspeções de código: promovendo padronização, segurança e clareza do software.
- Integração contínua (CI): cada nova versão do sistema passará por pipelines automáticas que verificam estabilidade, desempenho e cobertura de testes.
- Auditorias de requisitos: assegurando que cada requisito versionado esteja devidamente rastreável e sem duplicidade, evitando falhas de consistência.

Especificação da qualidade

A qualidade será especificada em conformidade com as dimensões de Garvin e os fatores de McCall, traduzidas para a prática do SGR:

- Confiabilidade: o sistema deve estar disponível e consistente em qualquer commit, sem perda de dados.
- Segurança (Integridade): o acesso ao sistema será controlado por autenticação segura, com níveis de permissão (gestor, analista, desenvolvedor).
- Eficiência: a busca de requisitos pela IA deve responder em tempo real, mesmo com grandes volumes de dados.
- Usabilidade: a interface do sistema será intuitiva, com fluxo simplificado para cadastro, commits e criação de branches.
- Manutenibilidade: cada requisito versionado estará isolado em branches e commits, facilitando correções sem afetar o restante do sistema.
- Portabilidade: o SGR será desenvolvido como aplicação web responsiva, podendo ser acessada em diferentes dispositivos e navegadores.

Metodologia de qualidade adotada

Adotaremos um processo de Garantia de Qualidade (QA) integrado ao desenvolvimento ágil, onde a qualidade não será verificada apenas no fim, mas a cada sprint. O ciclo da qualidade seguirá quatro atividades principais:

1. Planejamento da qualidade: definição de métricas (tempo de resposta da IA, taxa de falhas, tempo de recuperação em erro).
2. Controle da qualidade: revisões técnicas, auditorias de requisitos e testes regressivos.
3. Garantia da qualidade: relatórios de desempenho entregues ao Product Owner a cada

incremento, garantindo transparência e rastreabilidade.

4. Melhoria contínua: ajustes constantes baseados em feedback da Algomith e métricas coletadas no uso real.

Áreas de qualidade priorizadas

1. Infraestrutura: sistema escalável em nuvem, com redundância de dados e alta disponibilidade.
2. Segurança: criptografia de dados sensíveis e controle de acessos baseado em perfis.
3. Agilidade: respostas rápidas da IA e commits versionados em tempo real, alinhados ao modelo de fluxo do sistema.
4. Experiência do usuário: interface limpa, simples e responsiva, reduzindo curva de aprendizado.
5. Governança de requisitos: foco na rastreabilidade, versionamento e reuso como garantias de qualidade funcional.

Em resumo, a qualidade no SGR será multidimensional: técnica, operacional e perceptiva. O objetivo é que cada requisito, desde o seu commit inicial até sua reutilização em outro projeto, esteja respaldado por mecanismos que assegurem segurança, confiabilidade, performance e usabilidade, consolidando a confiança da Algomith no sistema e no processo.

Documentação

A documentação do Sistema de Gestão e Versionamento de Requisitos (SGR) será conduzida de forma integrada ao desenvolvimento, adotando boas práticas de clareza, padronização e automatização.

Todo o código-fonte será comentado de forma objetiva e padronizada, seguindo convenções da linguagem utilizada no projeto. Os comentários terão como objetivo explicar a finalidade de classes, funções e trechos críticos de lógica, evitando redundâncias desnecessárias. Essa prática garante que novos desenvolvedores compreendam rapidamente as implementações e facilita a manutenção do sistema a longo prazo.

O projeto adotará o Software Sphinx Documentation como ferramenta principal de geração de documentação. Através da anotação de docstrings nos módulos, classes e métodos, será possível produzir automaticamente manuais de referência técnica completos em PDF e JSON/HTML para indexação automatizada pela IA. Essa documentação incluirá:

- Descrição detalhada das funções e suas assinaturas.
- Exemplos de uso e integração.
- Referências cruzadas entre requisitos e funcionalidades implementadas.

Assim que a documentação for regenerada, o SGR receberá os arquivos exportados. A IA realizará a leitura semântica do conteúdo, extraindo descrições de classes, funções, parâmetros e relacionamentos. Esses dados serão vinculados aos requisitos versionados, garantindo que as implementações técnicas estejam sempre alinhadas às especificações registradas.

A documentação será tratada como parte essencial do desenvolvimento do projeto. Além de servir à equipe de desenvolvimento, ela será utilizada pela Algomith para auditoria, rastreabilidade e transferência de conhecimento, assegurando transparência e padronização no uso do SGR.

Conclusão

O desenvolvimento conceitual do SGR demonstra como um sistema especializado de versionamento e rastreabilidade pode transformar a gestão de requisitos em grandes organizações. Ao adotar princípios de versionamento semelhantes ao Git, mas adaptados ao domínio de requisitos, a proposta assegura consistência, padronização e reuso, reduzindo custos e aumentando a qualidade dos projetos de software. O uso de inteligência artificial agrega valor ao processo, oferecendo suporte cognitivo ao analista e automatizando tarefas de verificação e detecção de duplicidades.

A adoção de um modelo de processo evolucionário, iterativo e incremental, aliado a uma metodologia ágil híbrida (MAEC), garante entregas rápidas, feedback contínuo e capacidade de adaptação. Além disso, a integração da qualidade como princípio estratégico, com base em normas como a ISO/IEC 25010 e nas práticas de Pressman e Sommerville, assegura que o sistema seja robusto, seguro e alinhado às necessidades reais da Algomith.

Conclui-se que o SGR não apenas atende a uma demanda específica da empresa contratante, mas também se configura como um modelo inovador e replicável, capaz de inspirar outras organizações a adotarem práticas modernas de rastreabilidade, versionamento e governança de requisitos, consolidando o papel da qualidade como diferencial competitivo no desenvolvimento de software.

Referências

- SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.
- KOTONYA, Gerald; SOMMERVILLE, Ian. Requirements Engineering: Processes and Techniques. New York: Wiley, 1998.
- LAU, Kung-Kiu; WANG, Zheng. Software Component Models. In: IEEE Transactions on Software Engineering, v. 33, n. 10, p. 709–724, 2007.
- JACOBSON, Ivar; BOOCH, Grady; RUMBAUGH, James. The Unified Software Development Process. Boston: Addison-Wesley, 1999.
- LARMAN, Craig. Utilizando UML e Padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.
- BECK, Kent et al. Manifesto Ágil. 2001. Disponível em: <https://agilemanifesto.org/>. Acesso em: 20 ago. 2025.
- SCHWABER, Ken; SUTHERLAND, Jeff. Guia do Scrum: o guia definitivo para o Scrum – as regras do jogo. 2020. Disponível em: <https://scrumguides.org/>. Acesso em: 20 ago. 2025.
- WIEGERS, Karl E.; BEATTY, Joy. Software Requirements. 3. ed. Redmond: Microsoft Press, 2013.
- COCKBURN, Alistair. Writing Effective Use Cases. Boston: Addison-Wesley, 2001.
- MCCALL, James A.; RICHARDS, Paul K.; WALTERS, Gene F. Factors in Software Quality. Vol. I-III, Springfield: General Electric, 1977.
- ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. International Organization for Standardization, 2011.
- SPHINX. *Getting started — Sphinx documentation: Quickstart*. Disponível em: <https://www.sphinx-doc.org/en/master/usage/quickstart.html>. Acesso em: 16 set. 2025 às 20:00. Sphinx Documentation